Natalie Turner
November 10, 2025
IT FDN 110
Assignment05

Assignment 05: Advanced Collections and Error Handling

Intro

In this module, we applied dictionaries, JSON file types, and error handling to improve upon our student data program. Dictionaries are like lists, but they can be unordered as they contain key values that the data can be paired to, which makes working with dictionaries more clear and straightforward than working with lists. JSON files are formatted with key value pairs and values can be strings, numbers, booleans, etc. It can be used to represent complex or nested data structures, whereas CSV files can only hold plain text separated by commas. Error handling is used to manage errors that users of a program may introduce. These combined resulted in a program that was able to read in data from an existing file, collect user input, append the new data to the existing data and save it all to a file, and has built in error handling if the user inputs invalid responses.

Creating the Program

After creating the header for my program (not pictured), I defined the constants and variables that I would need. The key difference in this between assignment 04 and 05 is that student_data was defined with curly brackets {} instead of square brackets []. In assignment 4, student_data was a list of string values, but in this assignment student_data is a dictionary.

```
# Define the Data Constants
MENU: str = '''
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------
'''
# Define the Data Constants
FILE_NAME: str = "Enrollments.json"

# Define the Data Variables and constants
student_first_name: str = ''  # Holds the first name of a student entered by the user.
student_last_name: str = ''  # Holds the last name of a student entered by the user.
course_name: str = ''  # Holds the name of a course entered by the user.
student_data: dict = {}  # one row of student data
students: list = []  # a table of student data
file = None  # Holds a reference to an opened file.
menu_choice: str  # Hold the choice made by the user.
```

Figure 1.1: Defining constants and variables

Next, I wanted to create the initial .json file that was referenced with FILE_NAME. To do this, I used the file open function in write mode. Write mode when run the first time will create a new file, then I used file.close to save the file. After creating the initial file, I deleted these lines of code. Leaving them in my script would cause issues down the road because the write function clears the data that currently exists in the file each time it opens.

```
file = open('Enrollments.json', 'w')
file.close()
```

Figure 1.2: Creating the initial .json file

Next, I needed to enter my while loop and display the menu for users to see.

```
while True

    print (MENU)
    menu_choice = input("Enter your choice: ")
```

Figure 1.3: While loop and print the menu

Next, I set up my if/elif/else loop for the four menu options. For options 1, 2, and 3, I set up break placeholders that would allow my script to keep running without having finished the code for these options. For option 4, the break statement was used to allow users to exit my script once they were done entering and saving their data. I also included an else statement that was used as a catch all for invalid user input. It would print a statement letting users know that their choice is not valid and they need to try again.

```
    if menu_choice == '1':
        break
    elif menu_choice == '2':
        break
    elif menu_choice == '3':
        break
    elif menu_choice == '4':
        break
    else:
        print("Invalid choice. Please try again.")
```

Figure 1.4: if/else statement set up

I then went back to option one and began working on collecting user input and formatting it into a dictionary. I began with using the input function for the three variables

student_first_name, student_last_name, and course_name. I then created the dictionary variable student_data. Each of the variables student_first_name, student_last_name, and course_name were assigned a corresponding key variable. The key variables were enclosed in double quotes and the lines have a comma at the end to be consistent with JSON formatting. The dictionary was then appended to the students variable, which is a list.

```
if menu_choice == '1':
        student_first_name = input("Enter your first name: ")
        student_last_name = input("Enter your last name: ")
        course_name = input("Enter your course name: ")
        student_data = {
            "first_name": student_first_name,
            "last_name": student_last_name,
            "course_name": course_name,
        }
        students.append(student_data)
```

Figure 1.5 Collect user input and format to dictionary

After finishing out option 1, I moved on to creating the print statement for option 2. For this option, I used a for loop to iterate through the rows of data that had been stored in students, and pulled out each value using the dictionary key. I then used an f statement in the print line to display the data as comma separated string values.

```
elif menu_choice == '2':
    for student_data in students:
        student_first_name = student_data['first_name']
        student_last_name = student_data['last_name']
        course_name = student_data['course_name']
        print(f'{student_first_name}, {student_last_name}, {course_name}')
```

Figure 1.6 Print data in the file

I then moved on to saving the data to a JSON file in option 3. First, I had to open my file in write mode. I then used “json.dump” to add the data in the students variable into the file. Lastly, I used close() to save the data to the file.

```
elif menu_choice == '3':
    file = open(FILE_NAME, 'w')
    json.dump(students, file)
    file.close()
```

Figure 1.7: Saving data to the file

First Pass Testing

After finishing out the code for menu options, I tested my script by running through each menu option. I entered student data for option 1, then used option 2 to print out the data.

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your choice: 1
Enter your first name: Natalie
Enter your last name: Turner
Enter your course name: Python

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your choice: 2
Natalie, Turner, Python
```

Figure 2.1: Testing options 1 and 2

After verifying options 1 and 2, I used option 3 to save the the data I entered to the JSON file that I had created earlier. I then stopped my program with option 4.

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your choice: 3

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your choice: 4

Process finished with exit code 0
```

Figure 2.2: Testing options 3 and 4

In the initial test, I realized that there was no statement returned after choosing option 3 that confirmed to the user that the data was saved. I went back into my script to add a line for this, and then ran my script once again to verify that the statement was working.

Figure 2.3 Added print statement and retested option 3

Adding File Functionality

Next, I needed to go back to the beginning of my script and load the existing data from the JSON file. To do this, I opened the file with the read function and then used json.load to actually read the data into the script. In this step, I had to be sure that the file wasn't empty otherwise I would face an error.

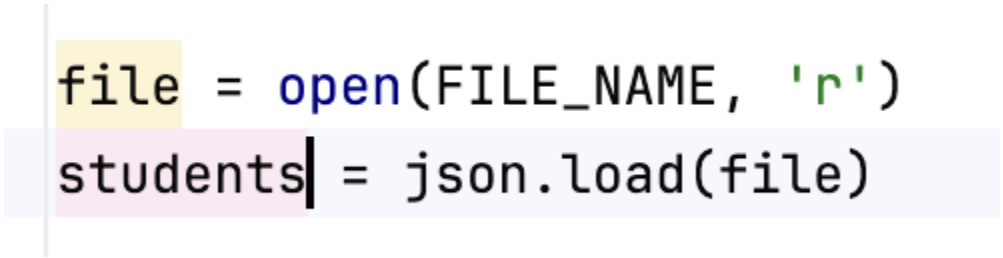

Figure 3.1: Loading existing data from the file

I then tested my script again to ensure that the data that existed in the file would be read into the script, and that the new user input would be appended. I walked through options 1 and 2 as I had done in the previous test. The main difference was that option 2 should have returned two rows of data - the row that already existed in the file and the new row that I added with option 1.

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your choice: 1
Enter your first name: Natalie
Enter your last name: Turner
Enter your course name: Python

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your choice: 2
Natalie, Turner, Python
Natalie, Turner, Python
```

Figure 3.2: Running options 1 and 2 again

Then I used option 3 to save the data to a file and used option 4 to close.

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your choice: 3
The following data has been saved to Enrolments.json: [{'first_name': 'Natalie', 'last_name': 'Turner', 'course_name': 'Python'}, {'first_na

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your choice: 4

Process finished with exit code 0
```

Figure 3.3 Testing options 3 and 4

I then checked the .json file itself to verify that the existing data and newly added data were saved as expected.

Figure 3.4 Verifying Enrollments.json

Error Handling

After completing and testing the main functionality script, I needed to go back through and add in error handling. The first place that I added error handling was for the part of the script where the existing data from the file is loaded in. To do this, I used a try/except loop. The first error that I considered was FileNotFoundError. Since the read function doesn't create a new file, it only reads an existing one, this error would occur if the called file didn't already exist. I wrote two print statements for this error. The first was a simple "file not found" and then second one referenced "e" which would display the actual error message to the user for a more technical audience.

```
try:
    file = open(FILE_NAME, 'r')
    students = json.load(file)
except FileNotFoundError as e:
    print("File not found")
    print("Technical error details:", e)
```

Figure 4.1 FileNotFoundError Handling

To test whether this error handling was working properly, I changed the FILE_NAME constant to “Enrollment.json” (no “s”) and reran the program.

```
'''
# Define the Data Constants
FILE_NAME: str = "Enrollment.json"

# Define the Data Variables and constants
student_first_name: str = ''  # Holds the first n
student_last_name: str = ''  # Holds the last nam
course_name: str = ''  # Holds the name of a cour
student_data: dict = {}  # one row of student dat
students: list = []  # a table of student data
file = None  # Holds a reference to an opened fil
menu_choice: str  # Hold the choice made by the u

try:
    file = open(FILE_NAME, 'r')
    students = json.load(file)
except FileNotFoundError as e:
    print("File not found")
    print("Technical error details:", e)
```

Figure 4.2: Updating FILE_NAME

Since “Entrollment**s**.json” is the only file that exists, this change should force an error.

```
/Users/natalieturner/Documents/PythonCourse/Module5/.venv/bin/python /Users/natalieturner/Documents/PythonCourse/Module5/Assignment05.py
File not found
Technical error details: [Errno 2] No such file or directory: 'Enrollment.json'
```

Figure 4.3: Error message for FileNotFoundError

I then chanted FILE_NAME constant back to “Enrollments.json” to ensure my script works properly in future runs. Next, I added a catch all error handling for issues other than FileNotFound. Again, I used two print statements to display messages to the user. The first was a very generic “Unexpected error” and the second again showed the actual error message for a more technical audience.

```
try:
    file = open(FILE_NAME, 'r')
    students = json.load(file)
except FileNotFoundError as e:
    print("File not found")
    print("Technical error details:", e)
except Exception as e:
    print("Unexpected error")
    print("Technical error details:", e)
```

Figure 4.4 Catch all error handling

The next part of the script that I added error handling was for menu choice 1. In this, I wanted to make sure that the user was entering the expected type of data for their first and last name. I again used a try/except loop, but this time also had to use if not statements to raise my own errors. For student_first_name and student_last_name, I used if not statements to check if entered characters were anything other than letters. If any characters were non-alphabetic, I used the raise function to throw a ValueError. This means that the entered data is not of the expected data type. The ValueErrors also displayed a message to the user letting them know that the entry can contain only letters. In the except line, I called the ValueError and used the print statement to again tell the user that the input is invalid, and used a second print statement to show the actual error message for the more technical audience. I also included a second except line to by a catch all for other unexpected issues. In this one, the first print statement just returned “unexpected error,” and the second line dgain showed the technical details of the error.

```python
if menu_choice == '1':
    try:
        student_first_name = input("Enter your first name: ")
        if not student_first_name.isalpha():
            raise ValueError ("First name must contain only letters")
        student_last_name = input("Enter your last name: ")
        if not student_last_name.isalpha():
            raise ValueError ("Last name must contain only letters")
        course_name = input("Enter your course name: ")
        student_data = {
            "first_name": student_first_name,
            "last_name": student_last_name,
            "course_name": course_name,
        }
        students.append(student_data)
    except ValueError as e:
        print("Invalid input")
        print("Technical error details:", e)
    except Exception as e:
        print("Unexpected error")
        print("Technical error details:", e)
```

Figure 4.5: Invalid input for menu choice 1

I then tested my error handling by running my script and intentionally entering invalid data. First I entered invalid data for the first name to check that the “invalid input” and “first name must contain only letters” messages would be displayed.

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your choice: 1
Enter your first name: 13
Invalid input
Technical error details: First name must contain only letters
```

Figure 4.6: Force error for first name

I then repeated the same thing but for the last name to ensure the error message was working here too.

```
Enter your choice: 1
Enter your first name: Natalie
Enter your last name: 12
Invalid input
Technical error details: Last name must contain only letters
```

Figure 4.7: Force error for last name

Lastly, I needed to add error handling for option 3. If there was any issue with saving the data to the file, this is when I would expect option 3 to run into an error. I used a try/except loop and used a catch all error. The first printed error statement would show “unexpected error” and the second one would show the more technical details.

```
elif menu_choice == '3':
    try:
        file = open(FILE_NAME, 'w')
        json.dump(students, file)
        file.close()
        print("The following data has been saved to Enrolments.json: " + f"{students}")
    except Exception as e:
        print("Unexpected error")
        print("Technical error details:", e)
```

Figure 4.7: Option 3 error handling

Testing in Pycharm

I finally was ready for the final test of my script in Pycharm. I first checked what data already existed in my .json file.

Assignment05.py Enrollments.json

```
1  [
2    {"first_name": "Natalie", "last_name": "Turner", "course_name": "Python"}
3  ]
```

Figure 5.1: Existing data in Enrollments.json

Then, I ran my script and used option 1 twice to enter data for two new students.

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your choice: 1
Enter your first name: Ary
Enter your last name: Pay
Enter your course name: Python

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your choice: 1
Enter your first name: Odie
Enter your last name: Dog
Enter your course name: Good boy school
```

Figure 5.2 Adding data for two new students

Next, I printed all existing data (new data and data that was already in the file) using option 2. Then I used option 3 to save data to file and verify that it returns a print statement that confirms the data is saved. Finally, I used option 4 to exit the program.

```
Enter your choice: 2
Natalie, Turner, Python
Ary, Pay, Python
Odie, Dog, Good boy school

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your choice: 3
The following data has been saved to Enrolments.json: [{'first_name': 'Natalie', 'last_name': 'Turner', 'course_name': 'Python'}, {'first_name': 'Ary', 'last

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your choice: 4

Process finished with exit code 0
```

Figure 5.3 Testing options 2, 3, and 4

There were no errors when running the script, the print statements looked correct, and Enrollments.json had been updated with the new data. This confirmed that the script was working as expected

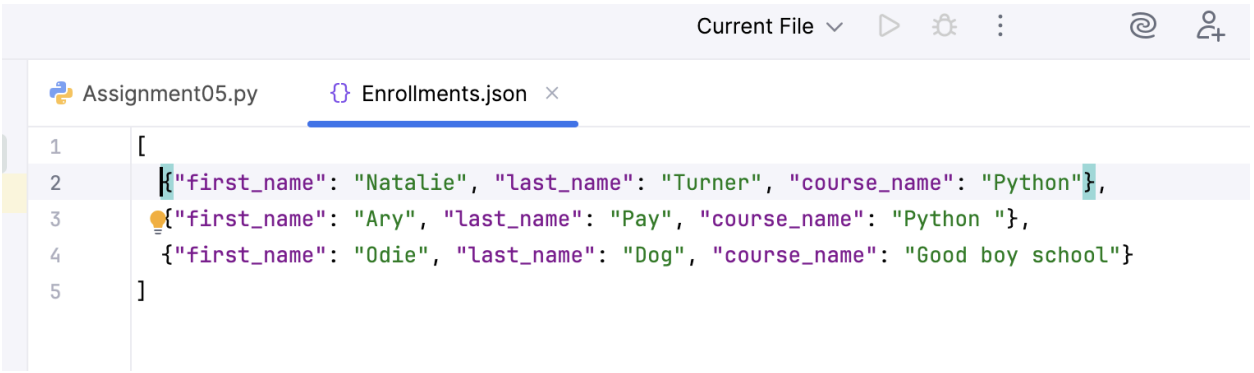

Figure 5.4: Enrollments.json updated with new data

Since I had tested the error handling as I was writing the code for it, I did not retest it again in Pycharm.

Testing in Terminal

Next, I tested my script in the terminal. Since I had just tested in Pycharm, there were already three entries in Enrollments.json that the next test would start with.

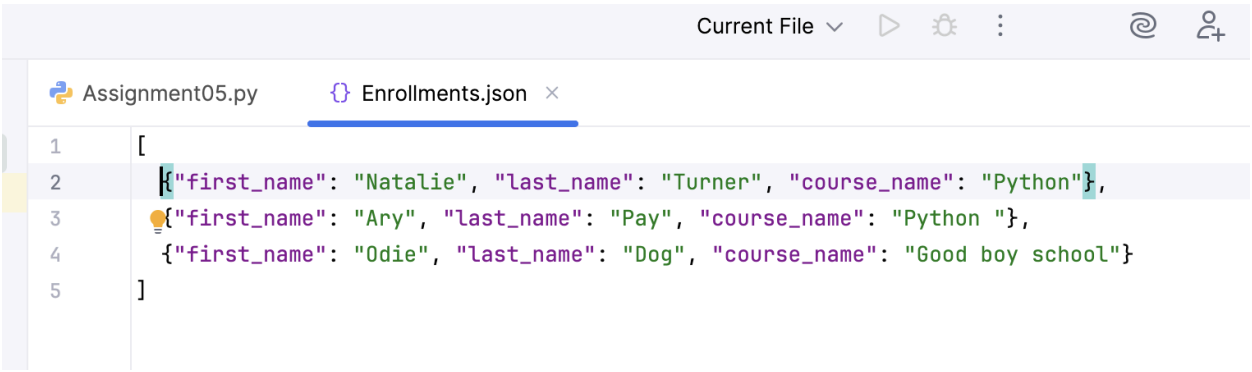

Figure 6.1: Starting data

I opened my terminal and used the change directory command and Python3 command to start my script. Then, I entered data for two additional students and used option 2 to print the new and existing data.

```
Last login: Mon Nov 10 15:53:51 on ttys000
natalieturner@Natalies-Air-2 ~ % cd /Users/natalieturner/Documents/PythonCourse/Module5/
natalieturner@Natalies-Air-2 Module5 % python3 Assignment05.py

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
-----------------------------------------

Enter your choice: 1
Enter your first name: Milo
Enter your last name: Dog
Enter your course name: Good boy school

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
-----------------------------------------

Enter your choice: 1
Enter your first name: Siena
Enter your last name: Neighbor
Enter your course name: Python

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
-----------------------------------------

Enter your choice: 2
Natalie, Turner, Python
Ary, Pay, Python
Odie, Dog, Good boy school
Milo, Dog, Good boy school
Siena, Neighbor, Python
```

Figure 6.2: Opening script and testing options 1 and 2

Then I used option 3 to save all of the data to a file, and option 4 to end the script.

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your choice: 3
The following data has been saved to Enrolments.json: [{'first_name': 'Natalie', 'last_name': 'Turner',
 'course_name': 'Python'}, {'first_name': 'Ary', 'last_name': 'Pay', 'course_name': 'Python '}, {'first
_name': 'Odie', 'last_name': 'Dog', 'course_name': 'Good boy school'}, {'first_name': 'Milo', 'last_nam
e': 'Dog', 'course_name': 'Good boy school'}, {'first_name': 'Siena', 'last_name': 'Neighbor', 'course_
name': 'Python'}]

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your choice: 4
natalieturner@Natalies-Air-2 Module5 %
```

Figure 6.3: Testing options 3 and 4

Finally, I opened Enrollments.json again to verify that the data that was originally in the file was still there and that the new data had been appended to the end.

Figure 6.4: Verifying Enrollments.json

This was the final verification that my script was working as expected.

GitHub

For this assignment, I needed to upload my work to GitHub. GitHub is a platform that can be used to create repositories of code and allow the community to collaborate. After creating my GitHub account, I created a new repository for module 5 that is visible to the public.

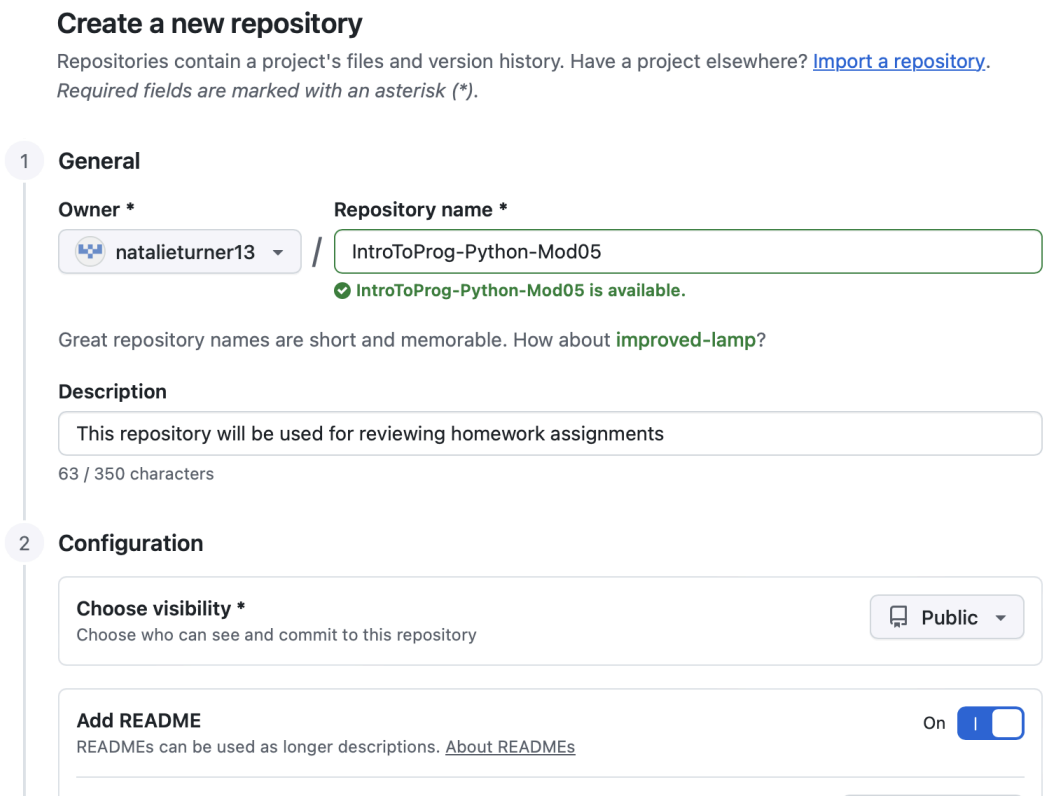

Figure 7.1: Creating module 5 repository

I then uploaded my python script and the starter Enrollments.json file to the repository. This document will also be uploaded once completed.

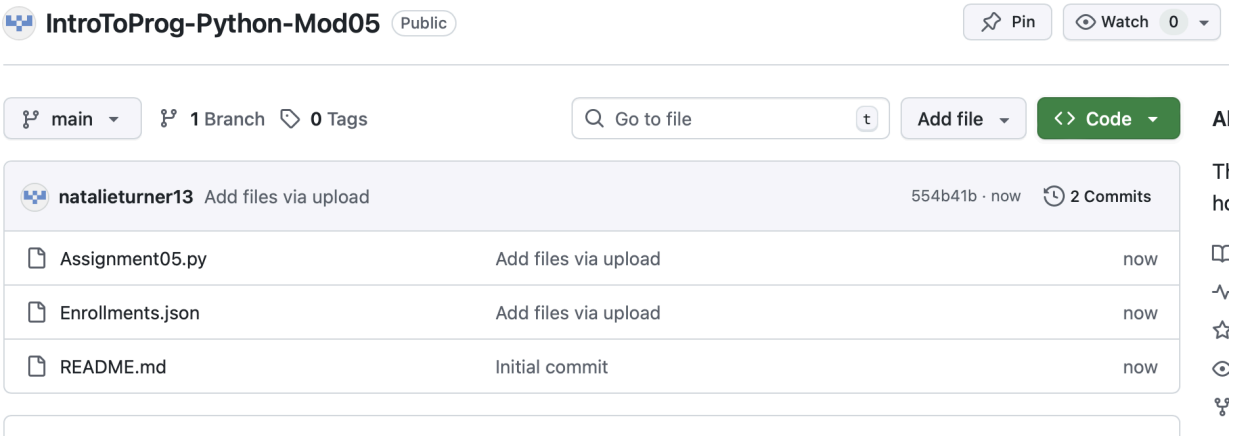

Figure 7.2: Uploading files to GitHub

Summary and Reflection

In this module, I learned about dictionaries, JSON files, error handling and GitHub. There was a lot of content covered, but it was mostly very similar to the material covered in previous modules. The part that I found the most challenging was error handling, as this was completely new to module 5. Being familiar with what errors python is able to return is important to understand before you can proactively account for those errors. I stuck to FileNotFoundError and ValueError as the two specific error types in my script, but I am sure that with more digging into the possibilities I would find more that could be added.